FAQ-02: Tagging — Sources, Standards, and Strategy

> **Series:** FAQ — Frequently Asked Questions | **Reference:** 02 — Tagging Sources, Standards, and Strategy | **Created:** May 2026 | **Last Updated:** 05/07/2026

Overview

Tagging is one of the most consequential decisions in a Dynatrace tenant — it drives ownership, alerting, IAM scope, automation targeting, cost attribution, and dashboard filtering. It is also one of the most commonly under-designed: teams accumulate tags from multiple sources without a coherent strategy, end up with inconsistent values across clouds, and discover the gaps only when an audit, a cost-allocation report, or an incident-routing rule fails.

This FAQ entry consolidates the four tag sources that flow into Dynatrace (OneAgent, Kubernetes, cloud providers, and legacy auto-tagging rules), the difference between **primary tags / primary fields** and ordinary tags, the standards that make a tagging strategy work across an estate, and the strategy themes that turn tag sprawl into a managed asset.

If you read only one section, read **§5 (Standards)** and **§6 (Strategy)** — together they answer most "how should we tag?" questions.

Table of Contents

Prerequisites

Requirement	Details
Audience	Platform team, SRE leads, FinOps stakeholders, security architects, anyone deciding how to model ownership/attribution in Dynatrace
Format	Decision-support document — presents sources, standards, and strategy; no hands-on lab
Related topic series	OPIPE (primary fields at source, OpenPipeline enrichment), CLOUD (per-provider field mappings), IAM (`dt.security_context` as boundary), AUTOM (config-as-code for any remaining tag rules), K8S (Kubernetes label propagation), ORGNZ (segments and bucket strategy that consume tags)

1. What "Tags" Mean in Dynatrace

"Tags" is an overloaded word. In a Dynatrace tenant, four distinct things commonly get called "tags":

Concept	Where it lives	When it shows up in Dynatrace
Cloud-provider tags	AWS / Azure / GCP resources	Via the Clouds app or legacy CloudWatch / Azure Monitor / GCP integrations; surface as `aws.tag.<key>` , `azure.tag.<key>` , `gcp.label.<key>` plus provider attributes (`aws.account.id` , `azure.resource_group` , `gcp.project_id`)
Kubernetes labels and annotations	K8s manifests / Helm / GitOps	Via DynaKube + OneAgent metadata enrichment; surface as `k8s.<resource>.label.<key>` (e.g., `k8s.pod.label.app` , `k8s.namespace.label.team`)
OneAgent host tags / primary tags	Set on the host at install or via `oneagentctl --set-host-tag=`	Ride on every signal at source — metrics, spans, logs, business events, Smartscape entities — including the sprint-1.337+ primary fields (`dt.security_context` , `dt.cost.costcenter` , `dt.cost.product`)
Auto-tagging rules *(legacy)*	Settings 2.0 (or legacy Configuration API) — rules that compute a tag at view time from other entity properties	Computed when an entity is viewed/queried; not present at ingest

These are not interchangeable. They differ in **where** they're set, **when** they're computed, **what** they propagate to, and **how** they're consumed downstream:

- **Where set:** changes to OneAgent host tags happen at the host's edge; cloud tags happen in the provider console; K8s labels happen in your manifests; auto-tagging rules happen in the Dynatrace Settings surface.
- **When computed:** the first three are set at ingest (the value rides on every signal); auto-tagging rules are computed at view/query time.
- **What they propagate to:** primary tags propagate to *every* signal type; auto-tagging applies primarily at the entity level (and not uniformly to logs, spans, business events).
- **How consumed:** Smartscape, dashboards, alerts/thresholds, IAM policies, OpenPipeline routing, segments — different consumers require different sources to be the source of truth.

The implication: **picking the right source for each dimension you tag on is more important than picking the right tag value**. A consistent value carried in the wrong source is harder to fix than a typo.

2. Primary Tags vs Primary Fields vs Custom Tags vs Auto-Tags

Inside the OneAgent surface, several distinct concepts share "tag"-adjacent vocabulary. Disambiguating them is essential:

Concept	Field shape	Example	Set how	Notes
Primary fields *(sprint-1.337+)*	Reserved Dynatrace key (`dt.*`)	`dt.security_context` , `dt.cost.costcenter` , `dt.cost.product`	OneAgent install / `oneagentctl --set-host-property=dt.security_context=<value>` (the documented primary-field assignment form); or via OpenPipeline enrichment for non-OneAgent sources	Top-level attributes on every signal at ingest. The recommended Gen3-first surface for boundary, cost, and ownership. Ride into Smartscape, IAM, and OpenPipeline routing without parse processors.
Primary tags *(sprint-1.337+)*	Customer-defined namespace (`primary_tags.<key>`)	`primary_tags.team` , `primary_tags.environment` , `primary_tags.app`	OneAgent install / `oneagentctl --set-host-tag=<value>` for free-form host tags; primary-tag namespace propagation is a Dynatrace convention layered on top	Also top-level on every signal. Use for dimensions that don't fit a reserved `dt.*` key.
Custom host tags *(legacy at-source tag)*	Plain key/value on the host	`Environment:prod` , `Owner:platform-team`	OneAgent install / `oneagentctl --set-host-tag=<value>` (no namespace prefix)	Pre-1.337 surface. Still works but lacks the primary-fields propagation guarantees; for new work prefer primary fields/tags.
Auto-tagging rules *(legacy, view-time)*	Tag conditions on entities (`Settings → Tags → Automatically applied tags`)	Rule: "if `host.name` matches `^prod-` then tag `Environment=prod` "	Settings 2.0 schema or legacy Configuration API	Computed when an entity is viewed; not present on ingest. **Avoid for new work** — see §7 for why.

> **`oneagentctl` syntax — primary fields vs tags:** The Dynatrace oneagentctl reference distinguishes between `--set-host-tag=<value>` (free-form host tags, including the `primary_tags.<key>` namespace by convention) and `--set-host-property=<key>=<value>` (reserved Dynatrace properties). The documented example is `oneagentctl --set-host-property=dt.security_context=easytrade_sec`. Use `--set-host-property` for `dt.*` reserved keys; use `--set-host-tag` for ordinary host tags and customer-defined `primary_tags.<key>` values.

Why primary fields/tags are the recommended Gen3-first default

Sprint-1.337 (April 2026) made `dt.security_context`, `dt.cost.costcenter`, and `dt.cost.product` plus customer-defined `primary_tags.*` first-class top-level attributes on **every** signal — metrics, spans, logs, business events, Smartscape entities — when set on the OneAgent at install time. Sprint-1.338 (May 2026) extended this to AWS Lambda via the `DT_TAGS` environment variable, which now also rides into logs and spans for serverless workloads.

Three reasons primary fields/tags are the correct default for new work:

1. **They flow without enrichment.** The value lands as a top-level field at ingest. No OpenPipeline parse processor, no auto-tagging rule, no view-time computation needed. DQL filters work directly: `filter dt.security_context == "team-a"`.
2. **They feed all downstream surfaces uniformly.** IAM `MATCH(dt.security_context)` boundary clauses, OpenPipeline `route` rules, Smartscape Ownership, and dashboard filters all see the same value.
3. **They are stable through time.** Set once at OneAgent install, the value rides through every host restart, every process restart, and every signal — no drift between dashboards and alerts.

Why auto-tagging rules are *not* the default

Auto-tagging rules compute a tag at view/query time from entity properties (host name, process name, etc.). They are convenient but introduce three problems:

- **Computed at view time, not ingest** — historical signals that predate a rule change get re-tagged retroactively from the *current* rule, not the rule that was in effect when the signal was generated. This breaks point-in-time analysis.
- **Don't propagate to logs, business events, or spans uniformly** — auto-tags primarily live on entities (hosts, services, process groups). DQL on `fetch logs` won't see the auto-tag without joining through the entity.
- **Couple a tag's value to a regex over a property that may change** — e.g., a rule that derives `Environment` from a host-name prefix locks the team into never renaming hosts.

Per Gen3-first guidance: tag at source via primary fields/tags, not at view time via auto-tagging rules. The migration path from a legacy tenant with auto-tagging rules in place is covered in §6 and §7.

3. The Four-Source Hierarchy

Tags arrive in Dynatrace from four sources. Each source is appropriate for some dimensions and inappropriate for others.

Source	Where set	Field surface in DQL	Best for
OneAgent — primary fields/tags	Host install / `oneagentctl --set-host-property=` (for `dt.*` reserved keys) and `oneagentctl --set-host-tag=` (for free-form tags including `primary_tags.<key>`)	`dt.security_context`, `dt.cost.costcenter`, `dt.cost.product`, `primary_tags.<key>`	Stable, host-level dimensions: security boundary, cost center, environment, team, app — anything that should ride on every signal at source
Kubernetes labels and annotations	K8s manifests / Helm / GitOps	`k8s.<resource>.label.<key>` (e.g., `k8s.pod.label.app`, `k8s.namespace.label.team`); annotations available as custom metadata at the process level	Dimensions that vary at the pod / workload / namespace level (more granular than the host) — application name, version, environment within a shared cluster
Cloud-provider tags	AWS / Azure / GCP console / IaC tooling	`aws.tag.<key>`, `azure.tag.<key>`, `gcp.label.<key>` plus provider attributes	The **source of record** for cost-allocation and compliance data when the cloud provider is the canonical owner of that information; useful as input to OpenPipeline enrichment that normalizes them into `dt.*` primary fields
Auto-tagging rules *(legacy)*	Settings 2.0 schema	View-time tag conditions on entities	Avoid for new work; acceptable only as a stop-gap on legacy tenants pending migration to primary fields

Propagation Depth — Why the Source Matters

Not all sources propagate to all signal types. This table is the load-bearing reason to choose source carefully:

Source	Metrics	Spans	Logs	Business Events	Smartscape Entities
OneAgent primary fields/tags *(sprint-1.337+)*	✓ at ingest	✓ at ingest	✓ at ingest	✓ at ingest	✓ as primary attribute
K8s labels/annotations	✓ (via DynaKube enrichment)	✓ (via DynaKube + OneAgent)	✓ (via OpenPipeline enrichment)	(rare)	✓ on K8s entity types
Cloud-provider tags	✓ (via cloud integration)	(limited; spans may not carry cloud tags directly)	(limited; depends on log ingestion path)	(limited)	✓ on cloud-resource entity types

Source	Metrics	Spans	Logs	Business Events	Smartscape Entities
Auto-tagging rules	(computed at view)	(computed at view)	(rare; not on raw log records)	(rare)	✓ at view time

If you tag at the OneAgent layer with primary fields, the value is on every signal type at ingest — DQL filters on `dt.security_context` work uniformly across `fetch logs`, `fetch spans`, `fetch bizevents`, `timeseries` queries, and entity queries. If you tag only via cloud-provider tags, the same query returns inconsistent results because the tag is on the cloud entity but not on every signal.

Propagation rule of thumb

> **For dimensions that need to filter / scope / route across signal types, tag at the OneAgent layer with primary fields/tags. For dimensions that are inherently scoped to a layer (K8s pod, AWS Lambda function), tag at that layer and rely on enrichment to surface them where needed.**

4. AWS / Azure / GCP — Per-Cloud Specifics

Cloud-provider tags are the source of record for cost-allocation, compliance, and ownership data when the cloud provider owns the resource lifecycle. They land in Dynatrace via the **Clouds app** (modern, GA for AWS, preview for Azure and GCP) or via legacy integrations (CloudWatch monitor for AWS, Azure Monitor for Azure, GCP integration for GCP).

AWS

AWS surface	Field in Dynatrace	Notes
EC2 / RDS / Lambda / ECS / EKS resource tags	`aws.tag.<TagKey>`	Tag keys preserved verbatim, including casing — `aws.tag.CostCenter` ≠ `aws.tag.costcenter`
Account ID	`aws.account.id`	Always present on AWS-sourced signals
Account alias	`aws.account.alias`	When set in IAM
Region	`aws.region`	Programmatic region code (e.g., `us-east-1`)
Resource ARN	`aws.arn`	Available on resources with discoverable ARNs

Sprint-1.337 — AWS Lambda `DT_TAGS`: Setting the `DT_TAGS` environment variable on a Lambda function injects primary fields (including `aws.arn`, `aws.region`, `aws.account.id`) and customer-defined primary tags into the Lambda's logs and spans, not just metrics. This closes the gap where serverless functions previously had partial tag propagation.

AWS integration path: prefer the **Clouds app** for new tenants — direct cloud connection without ActiveGate, GA for AWS. The legacy CloudWatch monitor remains supported but does not benefit from continued enhancement.

Azure

Azure surface	Field in Dynatrace	Notes
Resource tags (VMs, App Services, AKS, etc.)	`azure.tag.<TagKey>`	Tag keys preserved verbatim
Resource group	`azure.resource_group`	Useful as a coarse boundary if not tagged explicitly
Subscription	`azure.subscription_id`, `azure.subscription_name`	Always present on Azure-sourced signals
Region	`azure.region`	**Sprint-1.337 (OneAgent 1.337) — programmatic naming.** Region strings are now lowercase with no spaces (e.g., `eastus2`, not `East US 2`). Update DQL filters and dashboards comparing `azure.region` against display strings — use `in(azure.region, {"eastus2", "westus2"})`.

Azure integration path: Clouds app (preview) is the modern path; Azure Monitor integration via ActiveGate remains the production-stable option.

GCP

GCP surface	Field in Dynatrace	Notes
Resource labels (Compute Engine, GKE, Cloud Run)	`gcp.label.<LabelKey>`	GCP enforces lowercase + hyphens on label keys; values still arbitrary
Project	`gcp.project_id`	Always present on GCP-sourced signals
Region / zone	`gcp.region`, `gcp.zone`	Programmatic codes

GCP integration path: Clouds app (preview); legacy GCP integration via ActiveGate also supported.

Cross-Cloud Tag Naming Drift — A Concrete Problem

The same conceptual dimension lands with different keys across providers:

Concept	AWS	Azure	GCP
Cost center	`aws.tag.CostCenter`	`azure.tag.costCenter`	`gcp.label.cost_center`
Environment	`aws.tag.Environment`	`azure.tag.environment`	`gcp.label.environment`
Team / owner	`aws.tag.Owner`	`azure.tag.owner`	`gcp.label.team`
Application	`aws.tag.Application`	`azure.tag.app`	`gcp.label.application`

The casing alone (`CostCenter` vs `costCenter` vs `cost_center`) means a naive query has to enumerate every variant. Most teams discover this when they try to build a single "cost by team" dashboard that has to span clouds — and the dashboard is full of `coalesce(...)` that papers over the inconsistency.

The recommended fix is to normalize at ingest, not at query time. OpenPipeline enrichment processors can map `aws.tag.CostCenter` / `azure.tag.costCenter` / `gcp.label.cost_center` into a single canonical `dt.cost.costcenter` field, so DQL queries and IAM policies see one consistent surface regardless of cloud provenance. See OPIPE topic series for the worked enrichment-processor examples.

5. Tagging Standards — Taxonomy and Naming

A tag *strategy* is the source-of-truth and propagation decisions in §6. A tag *standard* is the taxonomy and naming convention that makes the strategy executable. Standards before tools — pick the dimensions and the names before configuring anything.

Recommended Dimensions to Tag

Most tenants benefit from tagging on these seven dimensions. Not every dimension needs to be a primary field; some can stay as ordinary tags or labels. The point is to decide *which* dimensions matter and to name them consistently.

Dimension	Purpose	Recommended source	Recommended key
Environment	Separate prod / nonprod / dev for thresholds, alerts, IAM scope	OneAgent primary tag at install	`primary_tags.environment` (values: `prod`, `nonprod`, `dev`)
Application / service	Identify which app a host or pod serves	OneAgent primary tag (host-level) or K8s label (pod-level)	`primary_tags.app` (host) or `k8s.pod.label.app` (pod)
Team / owner	Who owns this; drives notification routing	OneAgent primary tag at install	`primary_tags.team`
Cost center	FinOps attribution; cost-allocation reports	OneAgent primary field (preferred) or normalized from cloud tag via OpenPipeline	`dt.cost.costcenter`
Product / business line	Higher-level grouping above cost center	OneAgent primary field	`dt.cost.product`
Security context	IAM boundary for record-level / field-level access (the canonical Gen3 boundary)	OneAgent primary field at install	`dt.security_context`
Compliance / criticality	Regulatory or business-criticality scoping	OneAgent primary tag	`primary_tags.compliance` (values: `pci`, `pii`, `sox`, `none`) or `primary_tags.criticality` (`tier1`, `tier2`, `tier3`)

Optional further dimensions: deployment region, data residency, lifecycle phase. Add them only when there is a concrete consumer (a dashboard, an alert routing rule, an IAM policy) that would use them.

Reserved Dynatrace Keys

Some keys are reserved by Dynatrace. Don't redefine these — use them as-is, or use a different key:

Reserved namespace	Owned by	Don't use for
`dt.*`	Dynatrace platform (e.g., `dt.security_context`, `dt.cost.costcenter`, `dt.cost.product`, `dt.host.*`, `dt.smartscape.*`)	Custom dimensions — use `primary_tags.<key>` instead
`aws.*`, `azure.*`, `gcp.*`	Cloud-provider integrations	Custom dimensions; these are populated by the integration
`k8s.*`	Kubernetes integration	Custom dimensions; these are populated by DynaKube
`host.*`, `process.*`, `service.*`	OneAgent semantic dictionary	Custom dimensions

Naming Conventions

Element	Convention	Example	Anti-example
Key casing	lowercase, kebab-case if multi-word	`primary_tags.cost-center` (or use `dt.cost.costcenter`)	`primary_tags.CostCenter`, `primary_tags.cost_center` *(pick one and stick to it)*
Value casing	lowercase, kebab-case	`prod`, `nonprod`, `team-payments`	`Prod`, `NonProd`, `Team-Payments`
Value stability	use values that don't change frequently	`team-payments`, `app-checkout`	`release-2026-q2`, `incident-12345` *(metadata of the day)*
Spaces	never	`team-payments`	`team payments`
Reserved values	avoid `null`, `none`, `default`, empty string	`none-set` if you must	`null`, `""`
Cross-cloud normalization	one canonical key per dimension, regardless of provider	All cost-center tags resolve to `dt.cost.costcenter`	`aws.tag.CostCenter` and `azure.tag.costCenter` both queried separately forever

Industry Frameworks Worth Reading

If your organization doesn't have a tagging standard, these are reasonable starting points to adapt:

- **AWS Well-Architected — Tagging Best Practices** (docs) — practical taxonomy guidance including the 7 dimensions above
- **Azure — Develop your naming and tagging strategy** (docs)
- **GCP — Best practices for resource labels** (docs)

- **FinOps Foundation — Tagging best practices** — for the cost-attribution dimensions specifically

These frameworks broadly agree on the seven dimensions table above. Use them as the starting point and adapt names to your organization's existing conventions where they exist.

6. Strategy Themes

Standards say *what* to call things. Strategy says *who owns the value*, *what wins when sources conflict*, and *what to do when a source is missing*. Five themes:

6.1 One Source of Truth Per Dimension

For every dimension you tag on, designate exactly one authoritative source.

Dimension	Authoritative source	Why
Cost center	OneAgent primary field (or AWS Tag Editor for cloud-only resources, normalized via OpenPipeline)	One person's spreadsheet, not three
Environment	OneAgent primary tag at install	Stable per host, must be on every signal type
Team / owner	OneAgent primary tag at install	Tied to host placement, not pod scheduling
App name	OneAgent primary tag (host-level) **or** K8s label (pod-level) — pick one based on your topology	Mixing the two creates ambiguous joins
Security context	OneAgent primary field at install	Must be tamper-resistant; can't be a view-time computation

Mixing sources for the same dimension is the most common cause of tagging drift. If `team` is sometimes on the host and sometimes on the K8s pod label, every dashboard has to decide which to trust per-row.

6.2 Precedence and Conflict Resolution

When two sources both carry the same dimension, document which wins. A simple, stable precedence:

1. **OneAgent primary field/tag** (set explicitly at install) wins — most stable, most propagated
2. **K8s label** (when scoping inherently to pod/namespace) — next
3. **Cloud-provider tag** (when the cloud is the source of record for that resource) — next
4. **Auto-tagging rule** — last resort, only on legacy tenants

Document the precedence in your runbook. The runbook is consulted when an auditor asks "why does this report show team-A but the IAM policy targets team-B?"

6.3 Cardinality Control

Not every cloud tag should become a Dynatrace primary tag. Cloud accounts often accumulate dozens of tags per resource — automation tags, cost-allocation tags, compliance tags, deployment-pipeline tags. Surfacing them all into Dynatrace creates Smartscape clutter, dashboard-filter sprawl, and IAM-policy churn.

Curate which dimensions propagate as primary. A reasonable default: only the seven dimensions in §5 land as primary fields/tags. Other cloud tags can remain queryable as `aws.tag.<key>` / `azure.tag.<key>` / `gcp.label.<key>` for the ad-hoc queries that need them, without elevating them to first-class status.

6.4 Bare-Metal and On-Prem Fallback

Cloud tags don't exist on bare-metal hosts, on-prem VMs, or air-gapped infrastructure. **OneAgent host tags / primary tags are the universal floor** — they work the same way on every host regardless of provider.

Strategy implication: if your tenant spans cloud + on-prem, the *primary* tagging surface must be the OneAgent layer. Cloud-provider tags become a *secondary* enrichment for cloud-resident workloads. The opposite arrangement (cloud-tags-as-primary) leaves on-prem hosts with no consistent tag value.

6.5 Source-Side Enrichment Over View-Time Rules

When a dimension's authoritative source isn't already a Dynatrace-shaped key (e.g., `aws.tag.CostCenter` rather than `dt.cost.costcenter`), do the normalization **at ingest** via OpenPipeline enrichment processors, not **at view time** via auto-tagging rules.

Reasons:

- The normalized value lands on every signal at ingest (rather than being computed per-query)
- The OpenPipeline rule lives in source-controlled config (Terraform / Monaco / GitOps)
- A change to the rule is point-in-time; historical signals carry the value that was correct at the time they were ingested
- IAM policies and Smartscape see the normalized value uniformly

See OPIPE topic series for the worked enrichment patterns. The migration path from a legacy tenant: identify the auto-tagging rules that compute primary-dimension values, replace them with OpenPipeline enrichment, then deprecate the auto-tagging rules.

7. Anti-Patterns

Patterns that work in the short term and create rework in the long term. Each is followed by the recommended alternative.

7.1 Auto-Tagging Rule Sprawl

Symptom: the Settings → Tags → Automatically applied tags page has dozens of rules, often with overlapping conditions, that compute primary dimensions (environment, team, cost center) from regex over `host.name` or `process.name`.

Why it's a problem: rules are computed at view time; they don't propagate to logs / business events / spans uniformly; changing a rule retroactively re-tags historical signals; rules couple a tag's value to a property (host name) that may need to change for unrelated reasons.

Alternative: tag at source via OneAgent primary fields/tags. For cloud workloads, normalize cloud-provider tags via OpenPipeline enrichment. Migrate auto-tagging rules incrementally — replace one rule at a time, verify the at-source value matches the computed value, then disable the rule.

7.2 Treating Every Cloud Tag as Primary

Symptom: the Clouds app integration is configured to surface all `aws.tag.*` / `azure.tag.*` / `gcp.label.*` keys as Dynatrace primary tags. Smartscape is cluttered. Dashboard filter dropdowns show 80+ tag keys.

Why it's a problem: cloud accounts accumulate operational tags (deployment pipelines, automation jobs, ticket numbers) that aren't observability-meaningful. Surfacing them all elevates noise to first-class status.

Alternative: curate the dimensions that propagate as primary (the seven in §5). Other cloud tags remain queryable on `aws.tag.<key>` / `azure.tag.<key>` / `gcp.label.<key>` without first-class status.

7.3 Inconsistent Casing Across Clouds Without Normalization

Symptom: dashboards and DQL queries paper over `aws.tag.CostCenter` vs `azure.tag.costCenter` vs `gcp.label.cost_center` with `coalesce(...)`. The same dimension is queried differently in every report.

Why it's a problem: every new dashboard re-derives the normalization logic. Inconsistencies multiply. New team members don't know which key to query.

Alternative: normalize at ingest with OpenPipeline enrichment processors. One canonical `dt.cost.costcenter` field, regardless of cloud provenance.

7.4 Cost Center via Host-Name Regex

Symptom: an auto-tagging rule derives `CostCenter` from a host-name pattern like `^(?<cc>[a-z]{4})-prod-`.

Why it's a problem: hosts get renamed. Naming conventions evolve. The rule breaks silently. Worse — if the rule is wrong, the wrong cost center gets reported, often for months before someone notices the column doesn't sum to the total cloud bill.

Alternative: set `dt.cost.costcenter` explicitly at OneAgent install, sourced from the same authoritative system (CMDB, FinOps spreadsheet, AWS Tag Editor) as the cost-allocation report.

7.5 Relying on Cloud Tag for IAM Boundary Without Verifying Propagation

Symptom: an IAM policy uses `MATCH(aws.tag.SecurityContext)` as the boundary, assuming the AWS tag flows to every signal type the policy needs to scope.

Why it's a problem: AWS tags don't uniformly propagate to logs, business events, or all entity types. The policy works for some queries and silently fails-open for others. The user sees more data than the policy intended.

Alternative: use `MATCH(dt.security_context)` set at the OneAgent layer (or normalized via OpenPipeline from the cloud tag). The boundary then propagates to every signal type uniformly. See IAM topic series for the boundary-standardization pattern.

7.6 Tagging Metadata of the Day Into Long-Lived Primary Fields

Symptom: primary fields carry values like `release-2026-q2`, `incident-12345`, `feature-flag-payment-v2`. New values get added every sprint.

Why it's a problem: primary fields are designed for stable values that ride on every signal forever. Short-lived values churn the value-set, break dashboard filter dropdowns, and pollute IAM policy match conditions.

Alternative: primary fields/tags carry stable dimensions only (env, team, app, cost center, security context). Short-lived metadata (release version, incident ID, feature flag) belongs on **business events** — `fetch bizevents | filter event.release == "2026-q2"` works without polluting the host's primary tag set.

8. Final Recommendation

Tagging is foundational, not cosmetic. A coherent strategy pays back across every Dynatrace surface — Smartscape, dashboards, alerts, IAM, OpenPipeline, automation, cost attribution.

Four principles, in order of priority:

1. **Tag at source.** OneAgent primary fields/tags at install time are the universal floor. Cloud-provider tags supplement; auto-tagging rules retire.
2. **Use Dynatrace primary fields/tags as the canonical surface.** `dt.security_context`, `dt.cost.costcenter`, `dt.cost.product`, `primary_tags.<key>` — these are what dashboards, IAM, and routing should query against.
3. **Normalize cloud-provider tags via OpenPipeline enrichment.** Cross-cloud dimensions (cost center, environment, team) land in canonical `dt.*` fields at ingest, not at query time.
4. **Standards before tools.** Decide the seven dimensions, the naming convention, and the source-of-truth per dimension *before* configuring OneAgent installers, OpenPipeline processors, or IAM policies.

The cost of a bad tagging strategy is paid at every dashboard, every audit, every incident, every cost-allocation report — for as long as the tenant exists. The cost of a good one is paid once, up front.

Summary

Tagging in Dynatrace draws from four sources (OneAgent, Kubernetes, cloud-provider integrations, and legacy auto-tagging rules) that propagate differently and serve different purposes. Primary fields and primary tags (sprint-1.337+) are the recommended Gen3-first surface for stable dimensions because they ride on every signal at ingest. Standards (taxonomy + naming) come before strategy (source-of-truth + precedence + cardinality + fallback + source-side enrichment). The canonical pattern: tag at source via OneAgent, normalize cloud tags via OpenPipeline enrichment, and use `dt.*` primary fields as the surface every consumer queries.

Next Steps

- Inventory the dimensions your tenant currently tags on (across all four sources)
- Identify mismatches: dimensions tagged in multiple sources, dimensions tagged inconsistently across clouds, primary dimensions computed via auto-tagging rules
- Define the seven-dimension standard (or your organization's adapted version) before the next host onboarding wave
- Plan the OpenPipeline enrichment processors that normalize cross-cloud tags into `dt.*` canonical fields
- Cross-reference with FAQ-01 (host group naming strategy) and the IAM / ORGNZ / OPIPE topic series for the consuming patterns

9. Sources and References

Dynatrace official documentation:

- oneagentctl reference (Dynatrace docs) — `--set-host-tag` for free-form host tags and `--set-host-property` for reserved Dynatrace properties. The documented primary-field example is `oneagentctl --set-host-property=dt.security_context=easytrade_sec`. Cited in §2 and §3 for the correct syntax distinction
- Tags and metadata (Dynatrace docs) — the four sources of tags (manual, automated, cloud-imported, K8s-imported) and the distinction between tags and metadata. Cited in §1
- Host groups (Dynatrace docs) — host group concept and the relationship between host-group context and propagation; relevant to where host tags ride through topology. See FAQ-01 for the deep dive
- Dynatrace Clouds app (Dynatrace docs) — GA / preview status for AWS (GA — both new and classic connections), Azure (classic GA, new connections preview), GCP (classic GA, new connections planned). Cited in §4
- Dynatrace OpenTelemetry on AWS Lambda (Dynatrace docs) — AWS Lambda instrumentation and the `DT_TAGS` environment variable surface for primary-tag propagation into Lambda logs and spans

Cloud-provider tagging best practices (cited inline in §5):

- AWS Well-Architected — Tagging Best Practices — taxonomy guidance and the canonical seven-dimension tagging surface
- Azure — Develop your naming and tagging strategy — Microsoft's cloud-adoption-framework tagging recommendations
- Google Cloud — Best practices for resource labels — GCP label naming rules (lowercase, hyphens) and recommended dimensions
- FinOps Foundation — Tagging best practices — cost-attribution dimensions and tagging governance for FinOps maturity

Related topic series in this notebook collection (the consuming patterns):

- **OPIPE series** (`topics/opipe/`) — OpenPipeline enrichment processors for cross-cloud tag normalization and primary-field assignment at ingest
- **CLOUD series** (`topics/cloud/`) — per-provider integration deep dives (AWS, Azure, GCP) and field mapping reference
- **IAM series** (`topics/iam/`) — `MATCH(dt.security_context)` boundary patterns; the canonical Gen3 ABAC surface for record-level access
- **AUTOM series** (`topics/autom/`) — config-as-code for any remaining tag rules (Terraform / Monaco / GitOps)
- **K8S series** (`topics/k8s/`) — Kubernetes label propagation through DynaKube and OneAgent metadata enrichment
- **ORGNZ series** (`topics/orgnz/`) — segments and bucket strategy that consume tags as scoping inputs

Related FAQ entries:

- FAQ-01: Why you need a good Host Group naming strategy — the companion question on host-group boundaries; tagging-source-of-truth and host-group naming are decided together
- FAQ-03: OneAgent vs OpenTelemetry for Java/Scala — primary-field tagging assumes OneAgent presence; FAQ-03 covers when OneAgent is and is not the right tool

Source currency: Dynatrace doc shortlinks verified against the live `docs.dynatrace.com` site at the date in the metadata header. Sprint-1.337 / sprint-1.338 claims (primary fields, AWS Lambda `DT_TAGS`, Azure region

programmatic naming) are based on Dynatrace SaaS sprint-release notes and the `oneagentctl` command surface; verify against the latest release notes if the tenant is on an older sprint. Cloud-provider best-practice URLs confirmed live.
